

Hiddify Manager 面板代码级模块分析报告

分析日期：2026-06-14

分析对象：

- 外层仓库：`hiddify/hiddify-manager`
- 业务面板子仓库：`hiddify/HiddifyPanel`
- 官方文档入口：<https://hiddify.com/manager/>

本报告的目的不是教你怎么部署 Hiddify，而是拆解它“作为一个多用户代理面板”到底包含哪些模块、每个模块承担哪些功能，以及如果我们后续自研面板，哪些功能值得保留、哪些功能应该重做。

1. 总体结论

Hiddify Manager 不是一个单纯的 Web 面板。它更像一个“代理系统集成发行版”：

- `hiddify-manager`：负责系统安装、服务编排、Nginx/HAProxy/Xray/Sing-box/证书/防火墙/WARP/WireGuard 等运行时配置。
- `HiddifyPanel`：负责用户、管理员、域名、协议、订阅、API、流量统计、父子节点同步等业务控制面。
- 底层代理核心：同时使用 Xray-core 和 Sing-box。`core_type` 可选择 Xray 或 Sing-box，但 Sing-box 仍承担 Hysteria2、TUIC、WireGuard、统计等重要能力。
- 多用户订阅：支持浏览器用户页、普通链接订阅、Base64 订阅、Xray JSON、Sing-box JSON、Clash、Clash Meta、WireGuard 配置、SSH/sing-box 配置。
- 权限管理：支持 super admin、admin、agent 三层管理员和普通用户；管理员有最大用户数、最大活跃用户数、是否可新增子管理员等限制。
- VLESS + REALITY：是模型、订阅、Xray 模板、Sing-box 模板、HAProxy 分流都支持的一等协议。
- Hysteria2：不是 Marzban 那种旁路方案，而是 Hiddify 当前代码中的内置协议；通过 Sing-box Hysteria2 inbound 实现，多用户密码为用户 UUID。

对你要做的新面板来说，Hiddify 值得学习的地方是：

- 用户/管理员/套餐/流量模型。
- Domain + Proxy + Config 三层组合生成订阅的思路。
- 多客户端订阅适配。
- VLESS REALITY 与 Hysteria2 同时纳管。
- 订阅响应头里携带流量和到期信息。

不建议照搬的地方：

- HAProxy + Nginx + Xray + Sing-box + shell + sudo commander 的复杂系统耦合。
- Web 面板直接触发 root 级脚本的安全边界。
- Sing-box 用户启停主要依赖配置重载，而不是统一 agent 动态下发。
- 父子节点同步逻辑较粗，代码中也能看到 remote node 支持并不成熟。
- 管理界面基于 Flask-Admin，功能够用但产品体验和可维护性偏旧。

2. 代码快照和来源

本次分析使用了以下源码：

- `hiddify-manager`：浅克隆官方仓库，最新提交显示为 `7b1475fc7ad8930e5a66c451f3ea01937ca98f1b`，提交时间 `2026-05-29`。
- `HiddifyPanel`：由于子模块 clone 多次被网络中断，改用 GitHub code load ZIP 拉取 `main` 快照。
- 面板版本：`HiddifyPanel/pyproject.toml` 中显示 `12.3.3`。
- 官方 README 显示项目特性包括 Xray、Sing-box、Reality、Hysteria2、TUICv5、多用户、流量限制、时间限制、订阅页面、自动备份等。

关键源码路径：

- Manager 外层入口：`install.sh`、`apply_configs.sh`、`common/commander.py`
- Docker 部署：`Dockerfile`、`docker-compose.yml`
- HAProxy：`haproxy/*.j2`、`haproxy/fronts/*.pj2`、`haproxy/backends/*.pj2`
- Nginx：`nginx/*.j2`、`nginx/conf.d/*.j2`、`nginx/parts/*.j2`
- Xray 配置模板：`xray/configs/*.j2`
- Sing-box 配置模板：`singbox/configs/*.j2`
- Flask 应用入口：`hiddifypanel/base.py`
- 数据模型：`hiddifypanel/models/*.py`
- 管理后台：`hiddifypanel/panel/admin/*.py`
- 用户订阅页：`hiddifypanel/panel/user/user.py`
- REST API：`hiddifypanel/panel/commercial/restapi/v2/*`
- 订阅生成：`hiddifypanel/hutils/proxy/*.py`
- 流量统计：`hiddifypanel/panel/usage.py`、`hiddifypanel/drivers/*.py`
- 父子节点：`hiddifypanel/hutils/node/*.py`

3. 整体架构

3.1 组件分层

Hiddify 可以拆成五层：

1. 控制面 Web 层 - APIFlask / Flask - Flask-Admin - Flask-Classful - Flask-Babel - Flask-Session - SQLAlchemy
2. 数据层 - MariaDB / MySQL - Redis - SQLAlchemy ORM - Redis cache - Celery broker/result backend
3. 后台任务层 - Celery 每 60 秒更新用量 - 每 6 小时执行备份 - 用户超量/到期后触发代理核心移除用户
4. 代理核心层 - Xray-core - Sing-box - WireGuard - SSH Liberty Bridge - Telegram MTProxy - DNSTT - WARP 出站
5. 系统入口和服务编排层 - HAProxy: SNI / ALPN / path / CDN IP 分流 - Nginx: HTTP/gRPC/静态/面板/伪装站点 - acme.sh: 证书申请 - iptables/ip6tables: 防火墙规则 - systemd: 服务管理 - shell scripts: 安装、更新、状态、重启、配置渲染

3.2 Web 应用启动流程

`hiddifypanel/base.py` 中的 `create_app` 根据模式加载模块：

- web 模式：
- `base_setup`
- `panel.common`
- `panel.common_bp`
- `panel.admin`
- `panel.user`
- `panel.commercial`
- `panel.node`
- `celery`
- cli 模式：
- 数据库
- 日志 CLI
- 管理 CLI
- `celery` 模式：
- Celery app

Web 应用使用 `APIFlask`，OpenAPI 文档挂在 `/<proxy_path>/api` 前缀下。静态文件路径也带 `/<proxy_path>/static/`，这符合它把面板藏在随机路径后的设计。

3.3 配置下发流程

核心流程如下：

管理员修改配置/用户/域名/代理

- > 数据库更新
- > `hiddify.quick_apply_users` 或 `Actions.reinstall/apply`
- > `common/commander.py` 以 `sudo` 调用系统脚本
- > `hiddify all-configs` 读取数据库导出 JSON
- > `common/replace_variables.sh` 用 Jinja 渲染模板
- > 生成 Xray/Sing-box/HAProxy/Nginx/防火墙配置
- > `systemd` 重启/应用服务

Hiddify 的配置并不是一个单一 JSON，而是多服务模板渲染出来的一组配置文件。

3.4 用户访问流量路径

典型路径：

客户端

- > HAProxy :80/:443/UDP 443/协议专用端口
- > 按 SNI/ALPN/path/CDN IP 分流
- > Nginx 或 Xray 或 Sing-box
- > 对应协议 inbound
- > 出站 freedom/warp/blackhole

VLESS REALITY:

客户端 -> HAProxy SNI 分流 -> Xray/Sing-box REALITY inbound

Hysteria2:

客户端 UDP -> Sing-box hysteria2 inbound -> 出站

订阅:

客户端订阅请求

- > /<proxy_path_client>/<uuid>/
- > UserView 自动识别 User-Agent
- > 生成 sing-box / clash meta / xray / links 等格式

4. 数据模型模块

4.1 BaseAccount 基础账号

文件: `models/base_account.py`

字段:

- `uuid`
- `name`
- `username`
- `password`
- `comment`
- `telegram_id`
- `lang`

能力:

- 作为 `AdminUser` 和 `User` 的抽象基类。
- 支持 UUID 查询。
- 支持用户名密码查询。

- 支持密码更新。
- 支持基础 JSON 导出。
- 支持批量导入/更新。

注意点：

- 代码中密码是明文字段，未看到哈希处理。
- 快速配置历史上曾经偏向 UUID 链接免密码，后续加了 admin password，但模型层没有现代化密码安全抽象。

4.2 AdminUser 管理员

文件： `models/admin.py`

管理员模式：

- `super_admin`
- `admin`
- `agent`

字段：

- `mode`
- `can_add_admin`
- `max_users`
- `max_active_users`
- `parent_admin_id`
- `users`
- `usages`
- `sub_admins`

能力：

- 支持层级管理员。
- 支持下级管理员递归查询。
- 支持一个管理员只能看到自己和下级管理员的数据。
- 支持限制最大用户数。
- 支持限制最大活跃用户数。
- 支持删除管理员时把其用户和 usage 转移给当前管理员。
- 支持 owner/super admin 自动创建。

后台页面功能：

- 管理员 CRUD。
- 管理员分享链接。
- 管理员角色选择。
- 是否允许新增子管理员。
- 最大用户数限制。

- 最大活跃用户数限制。
- 在线用户统计。
- 下级管理员过滤。
- 新管理员必须设置密码。

4.3 User 普通用户

文件: `models/user.py`

用户模式:

- `no_reset`
- `monthly`
- `weekly`
- `daily`

字段:

- `uuid`
- `name`
- `usage_limit`
- `package_days`
- `mode`
- `start_date`
- `current_usage`
- `last_reset_time`
- `last_online`
- `added_by`
- `max_ips`
- `enable`
- `ed25519_private_key`
- `ed25519_public_key`
- `wg_pk`
- `wg_pub`
- `wg_psk`

能力:

- 流量上限。
- 套餐天数。
- 周期性重置。
- 首次连接后开始计时。
- 超流量自动不可用。
- 过期自动不可用。

- 启停开关。
- 管理员归属。
- WireGuard 用户密钥。
- SSH 用户密钥。
- JSON 导入导出。

后台页面功能：

- 用户 CRUD。
- 搜索 UUID / 名称。
- 修改用户名、备注、UUID。
- 设置流量包。
- 设置套餐天数。
- 设置重置周期。
- 重置流量。
- 重置套餐开始时间。
- 显示剩余天数。
- 显示当前用量进度条。
- 显示在线状态。
- 显示用户链接。
- Telegram 私信入口。
- 创建/更新/删除后同步到代理核心。

限制：

- `devices` 当前基本返回空，连接设备数/多 IP 限制看起来未完整实现。
- `max_ips` 字段存在，但实际 `User.is_active` 中相关限制被注释。

4.4 UserDetails 用户节点详情

文件： `models/user.py`

字段：

- `user_id`
- `child_id`
- `last_online`
- `current_usage`
- `connected_devices`

设计目的：

- 支持分节点统计用户用量。
- 支持分节点在线状态。
- 支持未来设备/IP 统计。

当前状态：

- 代码中多处显示“暂未真正使用”或返回空数组。
- 真正统计主要落在 `User.current_usage` 和 `DailyUsage`。

4.5 Domain 域名

文件： `models/domain.py`

域名类型：

- `direct`
- `sub_link_only`
- `cdn`
- `auto_cdn_ip`
- `relay`
- `worker`
- `fake`
- `special_reality_tcp`
- `special_reality_xhttp`
- `special_reality_grpc`
- `old_xtls_direct`
- `dnstt`
- 旧字段： `reality` 已标记 deprecated

字段：

- `domain`
- `alias`
- `sub_link_only`
- `mode`
- `cdn_ip`
- `grpc`
- `servernames`
- `show_domains`
- `download_domain`
- `extra_params`
- `resolve_ip`
- `child_id`

能力：

- 域名可用于直连、CDN、Relay、订阅专用、伪装、REALITY、DNSTT。
- 域名可设置别名，显示在订阅节点名称。
- 域名可限制展示哪些其他域名节点。

- 域名可指定下载域名，供 xhttp 等分离上传/下载场景使用。
- CDN 域名可配置 CDN IP 或自动优选 IP。
- REALITY 域名可设置 servernames。
- 每个域名有内部端口派生：
- Hysteria2: `hysteria_port + domain.id`
- TUIC: `tuic_port + domain.id`
- Naive: `naive_port + domain.id`
- Special Reality: `special_port + domain.id`
- DNSTT: `5400 + domain.id`

后台页面功能：

- 域名 CRUD。
- 域名格式校验。
- direct 域名必须解析到服务器 IP。
- CDN/relay/fake 域名不能直接解析到服务器 IP。
- Cloudflare API 自动新增/更新 DNS。
- CDN IP 格式校验。
- REALITY friendly 检查。
- REALITY 目标站 ASN 提醒。
- REALITY fallback/servername 兼容性警告。
- 自动申请证书。
- 删除域名时删除 Cloudflare DNS。

4.6 Proxy 协议记录

文件： `models/proxy.py`

协议枚举：

- `vless`
- `trojan`
- `vmess`
- `ss`
- `v2ray`
- `ssr`
- `ssh`
- `tuic`
- `hysteria`
- `hysteria2`
- `wireguard`
- `naive`
- `mieru`

- dnstt

L3/安全层枚举:

- tls
- tls_h2
- tls_h2_h1
- h3_quic
- reality
- http
- kcp
- ssh
- udp
- custom

传输层枚举:

- h2
- grpc
- faketls
- shadowtls
- restls1_2
- restls1_3
- WS
- tcp
- ssh
- httpupgrade
- xhttp
- custom
- shadowsocks
- udp

CDN/入口类型:

- CDN
- direct
- Fake
- relay

字段:

- name
- enable
- proto

- `l3`
- `transport`
- `cdn`
- `params`
- `child_id`

能力：

- 不是“一条 proxy 等于一个节点”，而是“协议组合模板”。
- 实际订阅节点由 Proxy + Domain + Config 动态组合。
- 支持按 child/node 分配置。
- `params` 可存放 xhttp download、headers、mode 等扩展参数。

4.7 Config 配置

文件：`models/config.py`、`models/config_enum.py`

配置类型：

- `BoolConfig`
- `StrConfig`

配置分类：

- `admin`
- `branding`
- `general`
- `proxies`
- `domain_fronting`
- `telegram`
- `http`
- `tls`
- `mux`
- `tls_trick`
- `ssh`
- `ssfaketls`
- `mieru`
- `shadowtls`
- `restls`
- `tuic`
- `hysteria`
- `ssr`
- `kcp`
- `hidden`

- `advanced`
- `too_advanced`
- `warp`
- `reality`
- `wireguard`
- `shadowsocks`
- `additional_configs`
- `dnstt`

重要配置能力：

- 核心选择： `core_type = xray/singbox`
- 国家： Iran / China / Russia / Others
- 语言： en/fa/ru/pt/zh/my
- HTTP/TLS 端口。
- VLESS/Trojan/VMess/Reality 开关。
- TCP/WS/gRPC/HTTPUpgrade/XHTTP 开关。
- Hysteria2 开关、端口、obfs、上下行 Mbps。
- TUIC 开关和端口。
- WireGuard 开关、端口、密钥、noise trick。
- Mieru 端口、握手、多路复用。
- DNSTT 密钥和 resolver。
- TLS Fragment、mixed case、padding、ECH。
- Mux、Brutal 参数。
- WARP 模式和站点列表。
- Telegram Bot。
- Branding。
- Additional configs： 追加 sing-box/xrayjson/URL。
- 订阅类型开关。
- proxy path / admin path / client path。

4.8 Child 节点

文件： `models/child.py`

节点模式：

- `virtual`
- `remote`
- `parent`

字段：

- `id`

- `name`
- `mode`
- `unique_id`
- `domains`
- `proxies`
- `boolconfigs`
- `strconfigs`
- `dailyusages`

能力：

- 每个 child 有自己的域名、代理、配置。
- Root child id 为 0。
- Virtual child 会复制 root 的 proxy/config，并生成默认 sslip.io 域名。
- parent/child 模式用于多面板同步。

当前限制：

- 后台 `NodeAdmin` 中创建 remote node 会直接报错：`Remote nodes are not supported yet!`
- 多节点能力更像半成品，不建议直接作为我们自研架构的蓝本。

4.9 DailyUsage 日用量

文件：`models/usage.py`

能力：

- 按日期、管理员、child 记录用量。
- Dashboard 用于绘制用量趋势。
- 统计每日在线用户数。

5. 管理后台模块

5.1 Dashboard

文件：`panel/admin/Dashboard.py`

功能：

- 首次安装时跳转 QuickSetup。
- 检查面板是否过期。
- 显示系统状态。
- 显示 Top 进程。
- 显示管理员维度用量历史。
- parent 模式下显示 child 节点。
- 检查 child 域名是否可访问。

- 提示默认 sslip.io 域名风险。
- 提示默认用户未删除。
- 提示 SSH 密码登录风险。
- 支持删除 child。

5.2 UserAdmin

文件： `panel/admin/UserAdmin.py`

功能：

- 用户列表。
- 用户创建、编辑、删除。
- 搜索 UUID/名称。
- 列表显示启停状态。
- 列表显示用户链接。
- 列表显示当前流量进度。
- 列表显示剩余时间。
- 列表显示最近在线。
- 列表显示所属管理员。
- 设置流量限制。
- 设置套餐天数。
- 设置周期模式。
- 重置流量。
- 重置套餐天数。
- 修改 UUID。
- 修改备注。
- Telegram 发送消息入口。
- 删除最后一个用户时阻止。
- 创建/修改/删除后调用 driver 增删用户。
- 创建/修改/删除后触发 `quick_apply_users`。
- parent 模式下通知 child 同步。

权限：

- super_admin、admin、agent 可访问。
- 用户归属受当前管理员递归权限影响。

5.3 AdministratorAdmin

文件： `panel/admin/AdministratorAdmin.py`

功能：

- 管理员列表。

- 管理员创建、编辑、删除。
- 支持 super admin / admin / agent。
- 设置是否可添加子管理员。
- 设置最大用户数。
- 设置最大活跃用户数。
- 设置新密码。
- 显示管理员专属链接。
- 显示在线用户进度。
- 显示总用户进度。
- 显示活跃用户进度。
- 下级管理员递归过滤。
- 防止低权限管理员创建更高权限账号。
- 删除管理员时调用模型层转移其用户。
- parent 模式下通知 child 同步。

5.4 DomainAdmin

文件: `panel/admin/DomainAdmin.py`

功能:

- 域名列表、创建、编辑、删除。
- 设置域名模式。
- 设置 alias。
- 设置 servernames。
- 设置 CDN IP。
- 设置是否解析 IP。
- 设置展示域名列表。
- 设置 download domain。
- 设置 extra params。
- 生成管理员访问链接。
- 域名/IP 解析校验。
- direct 模式校验解析 IP 是否等于服务器 IP。
- CDN/relay/fake 校验解析 IP 是否不同于服务器 IP。
- wildcard 只允许 CDN/auto CDN。
- Cloudflare DNS 自动写入/删除。
- REALITY 目标站可用性检查。
- REALITY ASN 警告。
- REALITY fallback/servername 兼容性警告。
- fake 域名自动填充 CDN IP。
- 删除最后一个域名时阻止。
- 需要证书的域名自动调用 `get_cert`。

- child 模式下同步域名到 parent。

5.5 ProxyAdmin

文件: `panel/admin/ProxyAdmin.py`

功能:

- 全局协议开关。
- 详细协议组合开关。
- 按 CDN/入口类型分组。
- 按协议分组。
- 对单个 Proxy 记录启停。
- 支持 wireguard/tuic/ssh/hysteria2/mieru 归到 other 分组。
- 更新后清理 proxy cache。
- child 模式下同步 proxy/hconfig 到 parent。
- 更新后提示 apply config。

5.6 SettingAdmin

文件: `panel/admin/SettingAdmin.py`

功能:

- 动态读取 ConfigEnum 生成配置表单。
- 按配置分类展示。
- 超级管理员可访问。
- 支持设置语言、国家、核心类型。
- 支持设置 WARP 模式。
- 支持设置 Telegram 实现。
- 支持设置 Mux 参数。
- 支持设置 TLS tricks。
- 支持设置 Hysteria 上下行 Mbps。
- 支持设置端口。
- 端口冲突检查。
- 80/443 特殊端口限制。
- proxy path 不允许重复。
- parent panel URL 校验。
- 自动注册 Telegram webhook。
- 自动判断是否需要 apply/reinstall/update。
- 品牌 HTML 文案使用 bleach 限制标签。
- 切换语言后刷新 Babel。
- child 模式下注册或同步 parent。

5.7 QuickSetup

文件: `panel/admin/QuickSetup.py`

步骤:

1. 选择管理员语言和国家。
2. 设置管理员密码。
3. 添加 direct 域名、可选 CDN 域名、decoy domain、站点分流开关。
4. 选择启用哪些协议。

功能:

- 首次安装引导。
- 国家选项包括 China。
- 直接域名必须解析到服务器。
- CDN 域名不能解析到服务器。
- 默认 sslip.io 域名清理。
- 完成后触发 reinstall/apply。

5.8 Actions

文件: `panel/admin/Actions.py`

功能:

- 查看日志列表。
- apply configs。
- reset/restart services。
- reinstall。
- update。
- status。
- 手动 update usage。
- 修改 REALITY 密钥。
- 获取所有公网端口。
- 生成日志页面。
- 查询域名 IP。
- 生成随机 REALITY friendly domain 候选表。

5.9 Backup

文件: `panel/admin/Backup.py`

功能:

- 导出完整配置 JSON。
- 恢复配置 JSON。

- 可选择恢复：
- 设置
- 用户
- 域名
- root admin
- 恢复后删除默认用户。
- 恢复后触发完整 reinstall。

Celery 中还会每 6 小时执行备份任务，并可通过 Telegram 把备份发给 super admin。

5.10 CommercialInfo / ProxyDetailsAdmin

文件：

- `panel/admin/commercial_info.py`
- `panel/commercial/ProxyDetailsAdmin.py`

功能：

- commercial 扩展入口。
- 在 Flask-Admin 中额外注册 Proxy 详情管理。
- 当前代码中商业授权校验基本返回 True，没有真正闭源授权限制。

6. 用户订阅和用户页面模块

文件： `panel/user/user.py`

6.1 入口和自动识别

用户访问：

```
/<proxy_path_client>/<uuid>/
```

行为：

- 如果是浏览器：显示新版用户主页。
- 如果是客户端：根据 User-Agent 自动返回合适订阅。

User-Agent 适配：

- Sing-box / HiddifyNext / SFI / SFA：返回完整 sing-box JSON。
- Clash-verge / Clash Meta / Stash / NekoBox / NekoRay / Pharos / hiddify-desktop：返回 Clash Meta。
- Clash / Stash：返回普通 Clash。
- v2rayNG / Streisand：在条件满足时返回 Xray JSON。
- Hiddify / FoXray / Fair / Shadowrocket / V2Box / Loon / Liberty 等：返回 Base64 链接订阅。

6.2 用户订阅端点

支持端点：

- `/auto`：自动选择配置。
- `/sub`：普通链接订阅。
- `/sub64`：Base64 链接订阅。
- `/xray`：Xray JSON 配置。
- `/singbox`：完整 Sing-box JSON。
- `/singbox-ssh`：SSH sing-box 配置。
- `/wireguard`：WireGuard 配置。
- `/clash`：普通 Clash。
- `/clashmeta`：Clash Meta。
- `/clash/proxies.yml`：只输出 proxies。
- `/clash/<type>.yml`：Clash 配置。
- `/full-singbox.json`：完整 Sing-box JSON。
- `/singbox.json`：旧 SSH sing-box 配置。
- `/all.txt`：全链接文本。
- `/offline.html`：离线页面。

6.3 订阅响应头

订阅响应会附带：

- `Subscription-Userinfo`
- `upload=0`
- `download=当前已用`
- `total=总流量`
- `expire=过期 Unix 时间`
- `profile-web-page-url`
- `support-url`
- `profile-update-interval`
- `profile-title`

这是多用户订阅分发中非常重要的一块，建议自研面板保留。

6.4 用户主页数据

`get_common_data` 会提供：

- 用户是否激活。
- 当前域名。
- 用户流量总量/已用。
- 到期时间。
- 剩余天数。

- 下次重置天数。
- 当前所有 hconfigs。
- 当前所有 domain 模式。
- 可用代理生成器。
- 用户 IP。
- 用户 ASN。
- 用户国家。
- Telegram bot。
- 是否启用 speedtest。
- 是否启用 telegram proxy。
- 用户 profile URL。

6.5 Additional configs

支持把额外配置合并进订阅：

- `additional_configs_urls`
- `additional_configs_singbox`
- `additional_configs_xrayjson`

URL 里可替换 `{{UUID}}`。

用途：

- 给用户订阅追加外部节点。
- 追加自定义 sing-box outbound。
- 追加自定义 xray JSON。

风险：

- 如果给普通管理员开放，可能引入配置污染或供应链风险。

7. 协议和订阅生成模块

核心文件：

- `hutils/proxy/shared.py`
- `hutils/proxy/xray.py`
- `hutils/proxy/xrayjson.py`
- `hutils/proxy/singbox.py`
- `hutils/proxy/clash.py`
- `hutils/proxy/wireguard.py`

7.1 核心设计

Hiddify 不是把“节点”直接存在数据库里，而是组合：

Domain + Proxy + Config + User + User-Agent -> 实际订阅节点

这带来两个好处：

- 一个域名可以自动派生很多协议。
- 一个协议模板可以应用到多个域名。

也带来复杂度：

- 用户看到的节点数量容易膨胀。
- 组合逻辑难调试。
- 每个客户端兼容性需要大量分支。

7.2 get_proxies

`get_proxies` 会根据全局开关过滤协议：

- DNSTT
- TUIC
- Mieru
- Naive
- WireGuard
- SSH
- Hysteria2
- Shadowsocks2022
- FakeTLS
- V2Ray plugin
- ShadowTLS
- SSR
- VMess
- VLESS
- Trojan
- HTTPUpgrade
- XHTTP
- WS
- gRPC
- TCP
- H2
- KCP
- REALITY
- QUIC
- HTTP proxy

还会根据是否存在 CDN/relay 域名过滤 CDN/relay proxy。

7.3 get_valid_proxies

这个函数会：

- 根据用户请求参数筛选 HTTP/TLS 端口。
- 按每个域名读取对应 child 的配置。
- 对每个启用的 Proxy 生成候选。
- 对 Hysteria2/TUIC/Naive/WireGuard/SSH/SS/Mieru/DNSTT 走特殊端口逻辑。
- 对 VLESS/Trojan/VMess 等按 http/tls 端口生成。
- 调用 `make_proxy` 生成最终节点。

7.4 is_proxy_valid

会校验：

- Naive 必须有有效证书。
- DNSTT 域名只能用于 DNSTT。
- 非 Mieru/DNSTT 必须有端口。
- REALITY proxy 只能和 REALITY domain 组合。
- REALITY domain 只能和 REALITY proxy 组合。
- special reality tcp/grpc/xhttp 必须匹配对应 transport。
- CDN proxy 只能和 CDN/auto CDN/worker domain 组合。
- 非 CDN proxy 不能用于 CDN domain。
- relay proxy 只能和 relay domain 组合。
- worker 不支持 gRPC。
- Trojan 必须配 TLS。

7.5 Hysteria2 订阅生成

普通链接：

```
hysteria2://<uuid>@<server>:<port>?hiddify=1&obfs=salamander&obfs-password=
<proxy_path>&sni=<sni>
```

如果是 fake 或不安全证书：

```
&insecure=1&allow_insecure=1
```

Sing-box outbound：

- type = `hysteria2`
- server / server_port
- password = 用户 UUID

- up_mbps / down_mbps
- obfs = salamander
- tls server_name / insecure

Clash Meta:

- type = hysteria2
- password = 用户 UUID
- obfs = salamander
- obfs-password
- sni
- skip-cert-verify

普通 Clash:

- 会过滤 Hysteria2, 因为普通 Clash 不支持。

7.6 VLESS + REALITY 订阅生成

Xray 链接字段:

- protocol: vless
- uuid: 用户 UUID
- encryption: none
- security: reality
- pbk: public key
- sid: short id
- fp: uTLS fingerprint
- flow: xtls-rprx-vision, 仅 tcp reality 场景。
- transport: tcp/grpc/xhttp

Sing-box outbound:

- type = vless
- uuid
- flow
- tls.enabled = true
- tls.reality.public_key
- tls.reality.short_id
- tls.utls.fingerprint
- transport tcp/grpc/xhttp

7.7 TUIC

支持:

- TUIC 链接。

- Sing-box TUIC。
- Clash Meta TUIC。
- password 和 uuid 都用用户 UUID。
- 默认 congestion_control = cubic。
- udp_relay_mode = native。
- zero_rtt_handshake = true。

7.8 WireGuard

支持：

- `wg://` 链接。
- WireGuard 配置文本。
- Sing-box endpoint/outbound。
- 用户级私钥、公钥、PSK。
- 服务器级公钥。
- 自动按用户 ID 派生 IPv4/IPv6。
- 可选 noise/fake packet trick。

7.9 SSH

支持：

- SSH 链接。
- Sing-box SSH 配置。
- 用户 ED25519 私钥。
- 服务器 host keys。
- SSH server 端口。

7.10 Shadowsocks / ShadowTLS / FakeTLS

支持：

- Shadowsocks2022。
- SS + obfs-local faketls。
- ShadowTLS v3。
- SS v2ray-plugin WebSocket。
- UDP over TCP。

7.11 Naive

支持：

- Naive TLS。
- Naive QUIC。
- 需要有效证书。

- 通过额外 header `hiddify-naive-secret` 携带 path。

7.12 Mieru

支持：

- TCP 端口范围。
- UDP 端口范围。
- multiplexing。
- handshake mode。

7.13 DNSTT

支持：

- DNSTT 域名模式。
- public key。
- resolvers。
- tunnel_per_resolver。
- keepalive / idle_timeout / client id / record type 等 extra params。
- 生成 DNSTT + socks 组合链接。

8. Xray / Sing-box 配置模板模块

8.1 Xray 模块

路径： `xray/configs/*.j2`

功能：

- 生成 Xray API： `127.0.0.1:10085`
- 启用 HandlerService / LoggerService / StatsService。
- 生成 routing。
- 生成 DNS。
- 生成 policy。
- 生成 stats。
- 生成各种 inbound：
 - api
 - socks main
 - kcp main
 - xtls main
 - reality main
 - dispatcher
 - dispatcher h2
 - new v10 inbounds

VLESS REALITY:

- Xray 模板支持 `special_reality_tcp`、`special_reality_grpc`、`special_reality_xhttp`。
- TCP 模式使用 `xtls-rprx-vision`。
- inbound listen 使用 abstract Unix socket: `@@realityin_<port>`。
- `realitySettings.dest` 指向域名 `:443`。
- `serverNames` 使用域名。
- private key / short IDs 来自 hconfig。
- sockopt 开启 `acceptProxyProtocol` 和 `tcpFastOpen`。

8.2 Sing-box 模块

路径: `singbox/configs/*.j2`

功能:

- 生成 Sing-box API: `127.0.0.1:10086`
- 启用 stats users。
- 启用 cache_file。
- 启用 monitoring。
- 生成 DNS/routing/outbounds。
- 生成 inbounds:
- Shadowsocks
- ShadowTLS
- socks main/auth
- VMess
- VLESS Reality
- TUIC
- Hysteria2
- Mieru
- Naive
- SSH

Hysteria2 inbound:

- `type = hysteria2`
- `listen = ::`
- `listen_port = domain.internal_port_hysteria2`
- up/down Mbps 来自配置。
- `obfs = salamander`, 可选。
- users 列表包含所有用户:
- `name = <uuid>@hiddify.com`
- `password = <uuid>`
- `masquerade` 指向 `http://<domain>:80/`

- TLS 开启, ALPN h3。
- 证书优先使用当前域名证书, 否则使用最后一个可用证书。

Sing-box REALITY:

- 支持 `special_reality_tcp`、`special_reality_grpc`。
- xhttp reality 仍倾向交给 Xray。
- `listen = 127.0.0.1:<special_port>`
- `proxy_protocol = true`。
- `tls.reality.handshake` 指向域名 443。
- `max_time_difference = 2h`。

8.3 用户列表方式

Xray VLESS 模板和 Sing-box Hysteria2 模板都会把所有有效用户直接写进配置:

- Xray VLESS clients: 每个用户 UUID 一个 client。
- Sing-box Hysteria2 users: 每个用户 UUID 一个 user/password。
- Sing-box stats users: 每个用户 UUID 加到 stats。

影响:

- 简单可靠。
- 用户多时配置文件会变大。
- Hysteria2 用户变更通常需要重载配置。
- Xray 可通过 API 动态增删, 但模板仍保留完整用户列表。

9. HAProxy / Nginx 入口分流模块

9.1 HAProxy

路径:

- `haproxy/haproxy.cfg.j2`
- `haproxy/fronts/in_tcpmode.cfg.pj2`
- `haproxy/fronts/sni_proxy.cfg.pj2`
- `haproxy/fronts/in_httpmode.cfg.pj2`
- `haproxy/backends/*.pj2`

能力:

- 监听 80、443、配置中的 http/tls ports。
- 处理 TLS SNI。
- 处理 ALPN h2/h3。
- 处理 CDN IP 段来源。
- 处理 path 分流。

- 处理 PROXY protocol。
- 转发到 Xray、Sing-box、Nginx、Panel。
- 对 FakeTLS、Telegram、ShadowTLS 做专用分流。
- REALITY special domain 按 SNI 分流到对应 inbound。
- panel 隐藏在随机 proxy path 下。
- generate_204 connectivity endpoint。

9.2 Nginx

路径：

- nginx/nginx.conf.j2
- nginx/conf.d/*.j2
- nginx/parts/*.j2

能力：

- Unix socket listener。
- HTTP/1.1 和 HTTP/2 分离。
- gRPC proxy。
- 面板反代。
- 静态文件。
- 伪装站点。
- proxy path 分流。
- speedtest。
- DoH 相关路径。

9.3 设计评价

优点：

- 功能很全。
- 支持复杂的 CDN/direct/reality/path 组合。
- 能把面板、订阅、代理入口隐藏在同一 443 体系。

缺点：

- 复杂度非常高。
- 自研面板第一版不建议照搬。
- 对调试和故障定位不友好。
- Hysteria2 这类 UDP 协议并不需要经过这套 HTTP/TLS 分流。

10. 流量统计和用户启停模块

核心文件：

- `panel/usage.py`
- `drivers/user_driver.py`
- `drivers/xray_api.py`
- `drivers/singbox_api.py`
- `drivers/wireguard_api.py`
- `drivers/ssh_liberty_bridge_api.py`
- `drivers/telemt_api.py`

10.1 Driver 聚合

`user_driver.py` 聚合多个 driver:

- XrayApi
- SingboxApi
- SSHLibertyBridgeApi
- WireguardApi
- TelemtApi

能力:

- 获取所有用户流量。
- 获取已启用用户。
- 添加用户。
- 删除用户。

10.2 XrayApi

能力:

- 使用 `xtlsapi.XrayClient('127.0.0.1', 10085)`。
- 从 Xray stats 查询用户用量。
- 动态 `add_client`。
- 动态 `remove_client`。
- 查询 inbound tags。
- 使用 xray CLI 查询 inbound users。

特点:

- Xray 动态用户增删相对完整。
- 支持不同 inbound tag 的 VLESS/Trojan/VMess/SS 映射。
- REALITY TCP 使用 `xtls-rprx-vision` flow。

10.3 SingboxApi

能力:

- 使用 `xtlsapi.SingboxClient('127.0.0.1', 10086)`。

- 从 Sing-box stats 查询用户用量。
- 从 `singbox/configs/01_api.json` 读取 stats users。

限制：

- `add_client` 是 `pass`。
- `remove_client` 是 `pass`。
- 这意味着 Sing-box/Hysteria2 的用户启停更多依赖配置生成与重载，不是动态 API。

10.4 WireguardApi

能力：

- 调用 `wg show hiddifywg transfer`。
- 用 Redis 保存上次统计。
- 根据用户 WireGuard pubkey 映射 UUID。
- 计算增量流量。
- 统计启用用户。

限制：

- `add/remove client` 为 `pass`。
- 依赖外部 WireGuard 配置生成。

10.5 用量更新任务

`usage.update_local_usage` 每分钟运行：

1. 从所有 enabled drivers 获取用量。
2. 调用数据库存储过程 `add_usage_json` 更新用户用量。
3. 生成/更新 DailyUsage。
4. 周期性检查是否需要重置用量。
5. 检查用户是否超流量/过期。
6. 如果状态变化，则 `add/remove client`。
7. 必要时 `quick apply users`。
8. 给绑定 Telegram 的用户发送到期/激活通知。

10.6 状态判定

用户 active 条件：

- 用户存在。
- `enable = true`。
- `usage_limit >= current_usage`。
- `remaining_days >= 0`。

已注释/未完整实现：

- 设备数/IP 数限制。

11. REST API 模块

11.1 Admin API

前缀：

```
/<proxy_path>/api/v2/admin/
```

端点：

- /me/
- /server_status/
- /admin_user/<uuid>/
- /admin_user/
- /log/
- /update_user_usage/
- /all-configs/
- /all-public-port/
- /user/<uuid>/
- /user/

功能：

- 当前管理员信息。
- 服务器状态。
- 管理员 CRUD。
- 用户 CRUD。
- 系统日志。
- 手动更新用户流量。
- 导出所有配置给 CLI。
- 返回公网端口。

认证：

- Hiddify-API-Key header。
- session。
- Basic auth。
- UUID path。

权限：

- super admin 可访问全部。

- admin/agent 只能访问自己或下级数据。

11.2 User API

前缀：

```
<proxy_path>/api/v2/user/  
<proxy_path>/<uuid>/api/v2/user/
```

端点：

- /me/
- /mtproxies/
- /all-configs/
- /short/
- /apps/

功能：

- 用户 profile。
- 当前用量/总流量/剩余天数。
- Telegram bot URL。
- 品牌信息。
- DoH URL。
- 速度测试开关。
- Telegram proxy 开关。
- 所有配置列表。
- 单节点链接列表。
- app 下载/引导信息。
- 短链。

11.3 Parent API

前缀：

```
<proxy_path>/api/v2/parent/
```

端点：

- /status/
- /register/
- /sync/
- /usage/

功能：

- child 状态检查。
- child 注册到 parent。
- child 上传 domains/proxies/hconfigs。
- parent 返回 users/admins。
- child 上传 usage。
- parent 计算增量 usage。

11.4 Child API

前缀：

```
/<proxy_path>/api/v2/child/
```

端点：

- /sync-parent/
- /register-parent/
- /status/
- /restart/
- /apply-config/
- /install/
- /update-usage/

功能：

- parent 要求 child 同步。
- child 注册 parent。
- parent 远程触发 child status/restart/apply/install/update usage。

11.5 Panel API

前缀：

```
/<proxy_path>/api/v2/panel/
```

端点：

- /info/
- /ping/

功能：

- 面板信息。
- 健康检查 PING/PONG。

12. 父子节点和多节点模块

核心文件：

- `hutils/node/parent.py`
- `hutils/node/child.py`
- `hutils/node/api_client.py`
- `hutils/node/shared.py`
- `panel/admin/NodeAdmin.py`

12.1 支持的模式

- Standalone：单面板。
- Parent：父面板。
- Child：子面板。
- Virtual child：同一个面板内的虚拟节点。

12.2 同步数据

Child 注册或同步时上传：

- `admin_users`
- `users`
- `domains`
- `proxies`
- `hconfigs`

Parent 返回：

- `users`
- `admin_users`

Usage 同步：

- child 上传所有用户当前 usage。
- parent 比较 parent 当前 usage，计算增量。
- parent 记入对应 child 的 DailyUsage。

12.3 API Client

使用：

- `requests`
- header: `Hiddify-API-Key`
- 最大重试 3 次。

12.4 当前成熟度

优点：

- 模型层支持 child_id。
- 配置、域名、代理都能分 child。
- API 具备 parent/child 基本同步。

问题：

- NodeAdmin 明确拒绝创建 remote node。
- 异步同步用 thread，缺少任务队列可靠性。
- token/权限模型不够现代。
- 错误恢复、节点离线、版本差异处理较弱。

对自研面板建议：

- 不直接复刻 parent/child。
- 改成 Control Plane + Node Agent。
- agent 使用独立 token、mTLS 或签名请求。
- 下发配置和上报状态都走明确版本化 API。

13. 安装、更新、服务编排模块

13.1 Docker

文件：

- Dockerfile
- docker-compose.yml

能力：

- Ubuntu 24.04 基础镜像。
- 暴露 80/443。
- privileged + NET_ADMIN。
- 依赖 MariaDB、Redis。
- 数据卷 /hiddify-data/。
- 容器内运行完整安装器。

评价：

- 易启动。
- 但 privileged 容器和 NET_ADMIN 权限很重。

13.2 install.sh

功能：

- 安装 Python venv。
- 安装 common/mysql/redis/panel。
- 读取面板配置。
- 渲染所有模板。
- 配置 system/firewall。
- 安装/运行：
 - Nginx
 - HAProxy
 - acme.sh
 - speedtest
 - DNSTT
 - Telegram proxy
 - FakeTLS
 - SSH proxy
 - WARP
 - Xray
 - Hiddify CLI
 - WireGuard
 - Sing-box
- 支持 apply_users 模式，只更新用户。

13.3 commander.py

功能：

- Web 进程通过 sudo 调用：
- apply
- install
- update
- status
- restart-services
- temporary-short-link
- get-cert
- update-usage
- apply-users
- update-wg-usage

安全评价：

- 已使用 `subprocess.run(cmd, shell=False)`，比拼接 shell 安全。
- 对 short link 和 domain 做了基本字符校验。
- 但 Web 面板整体仍拥有触发 root 运维脚本的能力，是高风险面。

13.4 restart/status/update

能力：

- 重启所有相关 systemd 服务。
- 查看 WARP 状态。
- 通过本地代理检查出口 IP。
- 查看服务状态。
- 自动更新。

14. 证书、防火墙、网络辅助模块

14.1 证书

路径： `acme.sh/*`

功能：

- 准备 acme.sh。
- 获取证书。
- 自签证书。
- 证书工具。

DomainAdmin 在需要有效 SSL 的域名变更后会调用 `get_cert`。

14.2 防火墙

路径： `common/run.sh.j2`

功能：

- 根据配置打开 TCP/UDP 端口。
- 打开 22、80、443。
- 打开 UDP 443。
- 打开 WireGuard 端口。
- 打开 Shadowsocks2022 端口。
- 打开每个域名派生出的 Hysteria2/TUIC/Naive 端口。
- 打开 Mieru TCP/UDP 端口。
- 支持防火墙 drop/accept 模式。
- 检查 SSH 密码登录并写入 MOTD 警告。
- 国家为 China 时设置 Asia/Shanghai 时区。

14.3 网络工具

路径： `hutils/network/*`

功能大类：

- 获取本机 IP。
- DNS 解析。
- Cloudflare API。
- 自动 CDN IP 选择。
- ASN/Country 查询。
- REALITY friendly domain 检查。
- ECH 信息获取。
- 用户真实 IP/ASN 识别。

15. Telegram Bot 模块

路径: `panel/commercial/telegrambot/*`

功能:

- 注册 Telegram bot webhook。
- 用户绑定 Telegram ID。
- 用户查询用量。
- 超量/过期/激活通知。
- 管理员备份文件发送。
- 用户页面显示 bot URL。

配置:

- `telegram_bot_token`
- `telegram_enable`
- `telegram_lib`
- `telegram_fakedomain`
- `telegram_adtag`

16. 备份和导入迁移模块

16.1 面板备份

导出内容:

- `childs`
- `users`
- `domains`
- `proxies`
- `admin_users`
- `hconfigs`

能力:

- 手动导出。
- 手动恢复。
- Celery 定时备份。
- Telegram 发送备份。

16.2 X-UI 导入

文件： `hutils/importer/xui.py`

功能：

- 读取 X-UI SQLite 数据库。
- 解析 inbound clients。
- 导入用户：
 - UUID
 - name/email
 - expiryTime
 - totalGB
 - current usage
 - telegram ID
 - enable
- 导入 REALITY 域名。

限制：

- 主要针对 X-UI 的数据结构。
- REALITY domain 模式代码里仍使用旧 `DomainType.reality`，与当前 special reality 类型存在历史包袱。

17. Hysteria2 支持能力专项分析

17.1 是否支持 Hysteria2

支持。代码中有完整链路：

- 模型枚举： `ProxyProto.hysteria2`
- 配置枚举：
 - `hysteria_enable`
 - `hysteria_port`
 - `hysteria_obfs_enable`
 - `hysteria_up_mbps`
 - `hysteria_down_mbps`
- 默认 Proxy：
 - `Hysteria2`
 - `Hysteria2 Relay`

- 订阅链接生成: `hutils/proxy/xray.py`
- Sing-box outbound: `hutils/proxy/singbox.py`
- Clash Meta: `hutils/proxy/clash.py`
- Sing-box inbound 模板: `singbox/configs/05_inbounds_4100_hysteria.json.j2`
- 防火墙开端口: `common/run.sh.j2`
- CLI 输出 domain:port: `hiddifypanel/panel/cli.py`

17.2 认证方式

Hysteria2 inbound 中:

```
{
  "name": "<uuid>@hiddify.com",
  "password": "<uuid>"
}
```

也就是:

- 用户 UUID 是 Hysteria2 密码。
- 多用户通过 Sing-box Hysteria2 users 列表实现。

17.3 端口策略

不是所有 Hysteria2 节点都用一个端口。

每个 domain 的端口为:

```
hysteria_port + domain.id
```

好处:

- 每个域名可对应不同 Hysteria2 inbound。

缺点:

- 端口数量增加。
- 防火墙、运营商阻断、客户端分发更复杂。

17.4 TLS/证书

Sing-box Hysteria2 inbound:

- TLS enabled。
- server_name = domain。
- ALPN = h3。
- 优先用当前域名证书。
- 如果当前域名证书不存在, 使用目录中最后一个 `.crt` 。

17.5 Obfs

支持 salamander:

- 开关: `hysteria_obfs_enable`
- password: 代码里使用 `proxy_path`

报告建议:

- 自研面板不要复用 `proxy_path` 当 obfs password。
- 应该给 Hysteria2 单独生成随机 obfs password。
- 用户订阅和 server config 同步使用同一 obfs secret。

17.6 速率参数

配置:

- `hysteria_up_mbps`
- `hysteria_down_mbps`

用于:

- 服务端 inbound。
- 客户端 outbound。

适合保留, 但应允许按节点、按套餐覆盖。

18. VLESS + REALITY 支持能力专项分析

18.1 是否支持

支持。代码中有完整链路:

- `ProxyL3.reality`
- `DomainType.special_reality_tcp`
- `DomainType.special_reality_grpc`
- `DomainType.special_reality_xhttp`
- `reality_enable`
- `reality_short_ids`
- `reality_private_key`
- `reality_public_key`
- `special_port`
- Xray REALITY inbound 模板。
- Sing-box REALITY inbound 模板。
- 订阅生成支持 reality fields。
- DomainAdmin 有 REALITY friendly 检查。

18.2 传输类型

支持：

- TCP Reality
- gRPC Reality
- XHTTP Reality

实现差异：

- Xray 支持 tcp/grpc/xhttp。
- Sing-box 模板中主要支持 tcp/grpc，xhttp reality 更倾向 Xray。

18.3 Flow

TCP Reality：

- `xtls-rprx-vision`

gRPC/XHTTP Reality：

- flow 为空。

18.4 REALITY 目标站检查

DomainAdmin 会：

- 检查域名是否 REALITY friendly。
- 检查 server 与目标域名 ASN 是否一致，若不一致给 warning。
- 检查 fallback/servernames 兼容性。

这部分非常值得自研面板借鉴。

18.5 密钥管理

Actions 提供：

- `change_reality_keys`

功能：

- 生成新的 X25519 key pair。
- 更新 private/public key。
- 提示 apply config。

建议：

- 自研面板中应支持按节点独立 REALITY key。
- 支持 short_id 多值。
- 支持密钥轮换策略和灰度切换。

19. 安全和可靠性观察

19.1 明显安全风险

1. Flask secret key 硬编码

base_setup.py 中:

```
app.secret_key = "asdsad"
```

这对生产系统不是好设计。即使 session 走 Redis, 固定 secret key 仍然不可取。

2. 密码字段未哈希

BaseAccount.password 是字符串字段, 登录时直接比较:

```
account.password != password
```

自研面板必须使用 Argon2id/bcrypt/scrypt。

3. Web 面板可 sudo 触发系统脚本

虽然 commander 使用 shell=False, 但权限边界很重。

4. 面板、订阅、代理入口高度混杂

Hiddify 通过随机路径隐藏面板, 但一台机器同时暴露太多服务。

5. 用户 UUID 同时承担多种认证角色

UUID 用于:

- 用户订阅地址。
- VLESS 用户 ID。
- Hysteria2 password。
- TUIC password。
- Trojan password。

这很方便, 但一旦订阅泄露, 所有协议凭据一起泄露。

19.2 可靠性风险

- Sing-box add/remove client 未动态实现。
- 用户数大时重载配置成本高。
- 父子节点同步靠 requests + thread, 缺少队列化、幂等和回滚。
- Jinja 模板和 shell 脚本分散, 排错难。
- Docker privileged 模式权限过高。
- 复杂 HAProxy/Nginx/Xray/Sing-box 交织导致故障面大。

19.3 兼容性风险

- 老 Clash 不支持 Hysteria2、TUIC、VLESS Reality 等，需要分流到 Clash Meta 或 sing-box。
- Hysteria2 依赖 UDP/QUIC，国内网络环境下需要备用 TCP Reality。
- REALITY xhttp 在不同核心/客户端兼容性差异较大。

20. 对自研面板的功能筛选建议

20.1 第一版必须有

用户和权限：

- 管理员登录。
- 密码哈希。
- super admin。
- admin/agent 可选。
- 用户 CRUD。
- 用户 UUID。
- 用户启停。
- 流量上限。
- 到期时间/套餐天数。
- 所属管理员。

节点和协议：

- 节点表。
- VLESS + REALITY 节点。
- Hysteria2 节点。
- Realm 中转节点。
- 节点分组。
- 节点启停。
- 节点健康状态。

订阅：

- 单用户订阅链接。
- sing-box JSON。
- Clash Meta YAML。
- 通用 links。
- v2rayN/Shadowrocket 可用链接。
- Subscription-Userinfo 响应头。
- 节点按用户权限过滤。

运行时：

- Node Agent。
- Agent 下发 Xray 配置。
- Agent 下发 Hysteria2 配置。
- Agent 上报流量。
- Agent 上报健康。
- 配置变更可回滚。

20.2 第二版应该有

- 多管理员层级。
- 套餐模板。
- 用户批量导入导出。
- Marzban/X-UI/Hiddify 导入。
- 节点测速。
- 晚高峰定时测速。
- 被墙检测。
- UDP 可用性检测。
- Realm 转发管理。
- 自动摘除故障节点。
- Telegram/邮件通知。
- 备份恢复。
- 审计日志。

20.3 可以暂时不做

- DNSTT。
- Mieru。
- Naive。
- Telegram proxy。
- WireGuard。
- WARP 出站。
- ShadowTLS/FakeTLS/SSR。
- Hiddify 的复杂 CDN/worker 模式。
- HAProxy 多层分流。
- XHTTP 高级下载域名拆分。
- 浏览器 PWA 用户主页。
- Commercial 扩展。

20.4 不建议照搬

- Flask-Admin UI。
- UUID 同时作为所有协议密码。
- proxy_path 作为 Hysteria2 obfs password。

- Web 直接 sudo 执行系统脚本。
- 用户列表全部写入所有配置后频繁重载。
- parent/child 面板同步模型。

21. 推荐的新面板模块边界

如果我们基于这份分析自研，建议模块如下：

21.1 Control Plane

负责：

- 管理员登录。
- 用户/套餐/权限。
- 节点/协议/中转。
- 订阅生成。
- 账务和流量。
- 操作审计。
- 配置版本。

21.2 Node Agent

负责：

- 安装/启动/停止 Xray。
- 安装/启动/停止 Hysteria2。
- 安装/启动/停止 Realm。
- 渲染本机配置。
- 上报流量。
- 上报进程状态。
- 上报端口连通性。
- 执行灰度切换。

21.3 Subscription Gateway

负责：

- 输出 sing-box。
- 输出 Clash Meta。
- 输出 v2rayN/Shadowrocket links。
- 根据用户套餐过滤节点。
- 根据客户端 User-Agent 输出最佳格式。
- 输出流量/到期 headers。

21.4 Node Monitor

负责：

- TCP 连接测试。
- UDP/QUIC 测试。
- TLS/REALITY 握手测试。
- Hysteria2 连通测试。
- 国内三网延迟/丢包测试。
- 自动标记故障节点。

21.5 Protocol Engine

负责抽象：

- VLESS REALITY。
- Hysteria2。
- TUIC 可选。
- Realm 中转。

每个协议应有：

- Server config generator。
- Client subscription generator。
- Traffic collector。
- Health checker。
- Credential rotation。

22. Hiddify 模块清单

下面是按“你筛选功能”角度整理的清单。

账号权限

- 管理员登录。
- 管理员 UUID。
- 管理员密码。
- super admin/admin/agent。
- 子管理员。
- 最大用户数。
- 最大活跃用户数。
- 普通用户。
- 用户 UUID。
- 用户启停。
- 用户备注。

- 用户语言。
- Telegram ID。

用户套餐

- 流量上限。
- 当前用量。
- 套餐天数。
- 开始日期。
- 剩余天数。
- 日/周/月/不重置。
- 重置流量。
- 重置套餐时间。
- 在线时间。
- DailyUsage。

协议

- VLESS。
- VLESS REALITY。
- Trojan。
- VMess。
- Shadowsocks2022。
- ShadowTLS。
- FakeTLS。
- SSR。
- SSH。
- TUIC。
- Hysteria2。
- WireGuard。
- Naive。
- Mieru。
- DNSTT。

传输

- TCP。
- WebSocket。
- gRPC。
- HTTPUpgrade。
- XHTTP。
- H2。
- H3/QUIC。

- KCP。
- UDP。

域名和入口

- Direct。
- CDN。
- Auto CDN IP。
- Relay。
- Worker。
- Fake。
- Sub-link only。
- Special REALITY TCP。
- Special REALITY gRPC。
- Special REALITY XHTTP。
- DNSTT domain。
- Download domain。
- Show domains。
- Alias。
- Servernames。
- Cloudflare DNS。
- 证书申请。

订阅格式

- Auto。
- Links。
- Base64 links。
- Xray JSON。
- Sing-box JSON。
- Clash。
- Clash Meta。
- WireGuard config。
- SSH sing-box。
- 单节点列表 API。
- 用户主页。
- 订阅流量 header。

系统运维

- 安装。
- 更新。
- 应用配置。

- 只应用用户。
- 重启服务。
- 状态检查。
- 查看日志。
- 防火墙。
- 证书。
- Speedtest。
- 备份。
- 恢复。
- Telegram 发送备份。

多节点

- Virtual child。
- Parent。
- Child。
- 节点状态检查。
- 注册 parent。
- 同步 domains/proxies/hconfigs。
- 同步 users/admins。
- 同步 usage。
- parent 触发 child restart/apply/install/status。

第三方/附加能力

- Cloudflare API。
- WARP。
- Telegram Bot。
- X-UI 导入。
- Additional configs。
- Auto CDN IP。
- REALITY friendly domain 检查。
- ASN 检查。
- ECH。
- TLS fragment。
- TLS padding。
- mixed case SNI。
- Mux。
- Brutal。

23. 对你的场景的建议

你现在的目标是：

- 多用户。
- 订阅分发。
- 权限管理。
- VLESS + REALITY。
- Hysteria2。
- Realm 中转。
- 中国内地环境下速度和稳定性都要兼顾。

所以我建议自研面板第一版不要复刻整个 Hiddify，而是吸收它的核心能力：

保留：

- 用户/管理员/流量/到期模型。
- VLESS REALITY。
- Hysteria2。
- Clash Meta / sing-box / links 订阅。
- Subscription-Userinfo。
- 节点分组。
- 流量统计。
- 健康检查。
- 备份恢复。

重做：

- 多节点架构。
- Agent。
- 安全模型。
- 密码和 token。
- Hysteria2 动态用户管理。
- Realm 中转管理。
- 监控和测速。

砍掉：

- Hiddify 的大量历史协议。
- 复杂 HAProxy/Nginx 分流。
- DNSTT/Mieru/Naive/Telegram proxy。
- WARP/WireGuard，除非后面确实需要。
- Flask-Admin 风格后台。

24. 最小可行功能集建议

如果你要我后续开始设计新面板，第一版 MVP 建议功能如下：

1. 管理员系统 - super admin 登录 - 密码哈希 - 操作日志
2. 用户系统 - 用户 CRUD - UUID/密码分离 - 流量上限 - 到期时间 - 启停
3. 节点系统 - 节点 CRUD - 节点 agent token - 节点健康状态 - 节点标签/分组
4. 协议系统 - VLESS REALITY TCP - Hysteria2 - Realm TCP 中转
5. 配置下发 - Agent 拉取配置 - 配置版本号 - 校验配置 - 应用/回滚
6. 订阅系统 - sing-box - Clash Meta - v2rayN links - Shadowrocket links - 流量 header
7. 流量统计 - Xray stats - Hysteria2/sing-box stats 或日志统计 - 用户累计用量 - 节点用量
8. 监控 - TCP 可用性 - UDP 可用性 - REALITY 握手 - Hysteria2 握手 - 延迟/丢包

这个 MVP 比 Hiddify 小很多，但更贴合你的业务。